Usage Examples — Common OSINT Workflows

These examples show how to use the system for real investigative workflows via the CLI and the REST API. The web UI mirrors all of these.

> Set up first:
> `bash`
> `osint-cli init-admin --username admin --password 'change-me'`
> `osint-cli create-user --username alice --password pw --role investigator --case CASE-2026-001`
> `osint-cli create-user --username bob   --password pw --role investigator --case CASE-2026-001`
> `osint-cli create-user --username val   --password pw --role viewer`

---

1. Social media evidence (with derived OCR artifact)

Scenario: Alice screenshots a suspect's recruitment post and later runs OCR to extract the text. The OCR output must be linked to the original.

Step 1 — capture the original screenshot

```
osint-cli add-evidence --user alice --case CASE-2026-001 \
  --title "Recruitment post by @suspect" \
  --source-type social_media --platform "twitter/x" \
  --source-url "https://x.com/suspect/status/123" \
  --tag recruitment --tag priority \
  suspect_post.png
# -> prints the new block_id, e.g. 9f3c...   and its file_hash
```

Step 2 — add the OCR text as a derived (versioned) block

Via the API (the CLI focuses on originals; derived blocks are an API/UI action):

```
TOKEN=$(curl -s -X POST localhost:3000/api/auth/login \
  -H 'Content-Type: application/json' \
  -d '{"username":"alice","password":"pw"}' | jq -r .token)

curl -s -X POST "localhost:3000/api/evidence/<PARENT_BLOCK_ID>/derived" \
  -H "Authorization: Bearer $TOKEN" \
  -F 'derivation_type=ocr' \
  -F 'tool=tesseract' \
  -F 'parent_file_hash=<ORIGINAL_FILE_HASH>' \
  -F 'metadata={"case_id":"CASE-2026-001","title":"OCR of recruitment
post","source_type":"document","classification":"UNCLASSIFIED"}' \
  -F 'files=@post_ocr.txt'
```

Now `GET /evidence/<PARENT_BLOCK_ID>` returns the OCR block under `derived[]`, giving you a verifiable lineage from screenshot → extracted text.

---

2. Document chain of custody

Scenario: A leaked PDF must be preserved with an unbroken custody record, and you need to prove months later it was never altered.

Ingest the document

```
osint-cli add-evidence --user bob --case CASE-2026-001 \
  --title "Leaked procurement contract" \
  --source-type document \
  --source-url "https://example.onion/files/contract.pdf" \
  --description "Obtained from public paste; SHA-256 recorded on ingest." \
  contract.pdf
```

Anytime later — prove integrity

```
# Verify the entire chain (signatures, links, Merkle roots)
osint-cli verify

# Verify this specific file still matches its on-chain hash
osint-cli verify-file <FILE_HASH>
# -> { "found": true, "intact": true, ... }
```

If anyone edits the stored PDF or a chain record, `verify` reports the exact block and `verify-file` reports `intact: false`.

Inspect the full custody record (incl. who viewed/exported it)

```
osint-cli show <BLOCK_ID>   # block + derived[] + access_log[]
```

---

3. Multi-investigator case

Scenario: Alice and Bob both contribute to `CASE-2026-001`; Val (viewer) reviews but cannot modify. Each investigator signs their own blocks.

Each investigator's contributions are independently signed

```
osint-cli add-evidence --user alice --case CASE-2026-001 \
  --title "Telegram channel export" --source-type messaging --platform telegram ex-
port.json

osint-cli add-evidence --user bob --case CASE-2026-001 \
  --title "Geolocated photo" --source-type geospatial photo.jpg
```

Every block stores the author's `collector_id` and `collector_pubkey`, and is signed with that investigator's private key. `verify` confirms each signature and (when keys are registered) flags any block whose key doesn't match the registered investigator — detecting impersonation.

Viewer access is read-only and audited

When Val opens evidence in the UI (or calls `GET /evidence/<id>`), an `access` block records the view. Val cannot create, export, or manage users:

```
# As Val, this is rejected with 403:
curl -s -X POST localhost:3000/api/evidence -H "Authorization: Bearer $VAL_TOKEN" ...
# -> { "error": "forbidden", "required_permission": "evidence:create" }
```

Review the team's audit trail

```
osint-cli audit                        # all access events
osint-cli audit --target <BLOCK_ID>    # everyone who touched one item
```

Managing case access (admin)

```
# Grant Bob access to a second case
curl -s -X PATCH localhost:3000/api/users/<BOB_ID> \
  -H "Authorization: Bearer $ADMIN_TOKEN" \
  -H 'Content-Type: application/json' \
  -d '{"assign_case":"CASE-2026-002"}'
```

---

4. Grouping an operation under one Merkle root

When you submit several files in **one** `POST /evidence` request (or one `add-evidence` call with multiple file arguments), they are committed together under a single **Merkle root**. This cryptographically binds the whole batch from one collection operation:

```
osint-cli add-evidence --user alice --case CASE-2026-001 \
  --title "Full thread capture" --source-type social_media \
  tweet1.png tweet2.png tweet3.png replies.html
# -> single block, merkle_root commits to all four file hashes
```

You can later prove any individual file belonged to that operation using the Merkle inclusion proof (`MerkleTree.proof` / `verify_proof` in `core/merkle.py`).

Tips

- **Backups:** copy `data/chain/chain.jsonl` off-host regularly; it is the source of truth and is human-readable.
- **Key portability:** an investigator's public key (`osint-cli export-pubkey --user alice`) can be shared so external parties can verify their signatures.
- **Offline ingest:** set `OSINT_NTP_ENABLED=false` in the field; blocks clearly mark the timestamp authority as `local-system-clock` .